Breakpoint model

1 Subject specific, single breakpoint, single slope

1.1 Direct optimisation

Consider a response variable Y and an explanatory variable X related by:

$$Y = \beta X - \beta(X - \psi)_+ + \varepsilon$$

where $(\beta, \psi) \in \mathbb{R}^2$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, and $(x)_+ = x$ if $x > 0$ and 0 otherwise. Introduce $\Theta = (\beta, \psi, \sigma^2)$, we can express the likelihood relative to n iid observations as:

$$\mathcal{L}(\Theta) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(Y_i - \beta X_i + \beta(X_i - \psi)_+)^2}{2\sigma^2}\right)$$

and the log-likelihood as:

$$\ell(\Theta) = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \sum_{i=1}^{n} \frac{(Y_i - \beta X_i + \beta(X_i - \psi)_+)^2}{2\sigma^2}$$

Maximizing the likelihood with respect to $\Theta_\mu = (\beta, \psi)$ is equivalent to minimizing the residual sum of squared (RSS):

$$RSS(\Theta_\mu) = \sum_{i=1}^{n} (Y_i - \beta X_i + \beta(X_i - \psi)_+)^2$$

One approach to estimate the model parameters is to directly minimize this log-likelihood using, for instance grid search. This may be numerically inefficient but does not require any smoothness assumption on the likelihood.

- this is what `optim.lmbreak_NLMINB` in the lmbreak package does (set argument `control` to `list(optimizer = "nlminb")`). `nlminb` seems to employ a quasi Newton method though, some does require some smoothness.

1.2 Gradient descent

One can re-write the residual sum of squares as:

$$RSS(\Theta_\mu) = \sum_{i=1}^{n} \varepsilon_i(\psi)^2 = Y(I - H(\psi))(I - H(\psi))^\intercal Y^\intercal = Y(I - H(\psi))Y^\intercal$$

where $H(\psi) = X(\psi)\left(X^\intercal(\psi)X(\psi)\right)^{-1} X^\intercal(\psi)$ is the hat matrix. It is idempotent and so is $I - H$ where I denote the identity matrix. Therefore a possible derivative w.r.t. ψ of the RSS is:

$$\begin{aligned}
\frac{\partial RSS(\Theta_\mu)}{\partial \psi} =& - Y\frac{\partial H(\psi)}{\partial \psi}Y^\intercal \\
=& - Y\frac{\partial X(\psi)}{\partial \psi}\left(X^\intercal(\psi)X(\psi)\right)^{-1} X^\intercal(\psi)Y^\intercal \\
&+ YX(\psi)\left(X^\intercal(\psi)X(\psi)\right)^{-1}\left(\frac{\partial X^\intercal(\psi)}{\partial \psi}X(\psi) + X^\intercal(\psi)\frac{\partial X(\psi)}{\partial \psi}\right)\left(X^\intercal(\psi)X(\psi)\right)^{-1} X^\intercal(\psi)Y^\intercal \\
&- YX(\psi)\left(X^\intercal(\psi)X(\psi)\right)^{-1}\frac{\partial X^\intercal(\psi)}{\partial \psi}Y^\intercal
\end{aligned}$$

The first and last term being scalar and transpose of each other they are equal. Similarly the two middle terms are also scalar and transpose of each other so equal. Therefore:

$$\begin{aligned}
\frac{\partial RSS(\Theta_\mu)}{\partial \psi} =& - 2Y\frac{\partial X(\psi)}{\partial \psi}\left(X^\intercal(\psi)X(\psi)\right)^{-1} X^\intercal(\psi)Y^\intercal \\
&+ 2YX(\psi)\left(X^\intercal(\psi)X(\psi)\right)^{-1}\left(X^\intercal(\psi)\frac{\partial X(\psi)}{\partial \psi}\right)\left(X^\intercal(\psi)X(\psi)\right)^{-1} X^\intercal(\psi)Y^\intercal \\
=& 2Y\left(-\frac{\partial X(\psi)}{\partial \psi} + X(\psi)\left(X^\intercal(\psi)X(\psi)\right)^{-1} X^\intercal(\psi)\frac{\partial X(\psi)}{\partial \psi}\right)\left(X^\intercal(\psi)X(\psi)\right)^{-1} X^\intercal(\psi)Y^\intercal \\
=& 2Y\left(X(\psi)\left(X^\intercal(\psi)X(\psi)\right)^{-1} X^\intercal(\psi) - I\right)\frac{\partial X(\psi)}{\partial \psi}\left(X^\intercal(\psi)X(\psi)\right)^{-1} X^\intercal(\psi)Y^\intercal
\end{aligned}$$

So it only remains to evaluate the (sub-)derivatives of $X(\psi)$:

$$\frac{\partial X(\psi)}{\partial \psi} = \frac{\partial (X - \psi)\mathbb{1}_{X>\psi}}{\partial \psi} = -\mathbb{1}_{X>\psi} + \mathbb{1}_{X=\psi}\delta$$

where $\delta \in [0, 1]$.

So a second approach to estimate the model parameters is to perform a gradient descent based on sub-derivatives

- this is what `optim.lmbreak_BFGS` in the lmbreak package does (set argument `control` to `list(optimizer = "BFGS")`). δ is then randomly sampled in a uniform distribution whose support is $[0, 1]$.

Note: another approach to 'profile-out' the β, is to first plug-in the OLS estimator:

$$\tilde{\beta}(\psi) = \frac{\sum_{i=1}^n X_i Y_i - (X_i - \psi)_+ Y_i}{\sum_{i=1}^n (X_i + (X_i - \psi)_+)^2} = \frac{\sum_{i=1}^n \mathbb{1}_{X_i \leq \psi} X_i Y_i}{\sum_{i=1}^n \mathbb{1}_{X_i \leq \psi} X_i^2}$$

and then minimize w.r.t. ψ:

$$\ell(\psi) = \sum_{i=1}^n (Y_i - \tilde{\beta}(\psi) X_i + \tilde{\beta}(\psi)(X_i - \psi)_+)^2$$

So the first 'derivative' should solve:

$$\begin{aligned}
0 =& -2 \sum_{i=1}^n \left[\frac{\partial \tilde{\beta}(\psi)}{\partial \psi} (X_i - (X_i - \psi)_+) - \tilde{\beta}(\psi) \frac{\partial (X_i - \psi)_+}{\partial \psi} \right] (Y_i - \tilde{\beta}(\psi) X_i + \tilde{\beta}(\psi)(X_i - \psi)_+) \\
0 =& \frac{\partial \tilde{\beta}(\psi)}{\partial \psi} \sum_{i=1}^n (X_i - (X_i - \psi)_+)(Y_i - \tilde{\beta}(\psi) X_i + \tilde{\beta}(\psi)(X_i - \psi)_+) \\
& - \tilde{\beta}(\psi) \sum_{i=1}^n \frac{\partial (X_i - \psi)_+}{\partial \psi} (Y_i - \tilde{\beta}(\psi) X_i + \tilde{\beta}(\psi)(X_i - \psi)_+)
\end{aligned}$$

Noticing that $\tilde{\beta}(\psi)$ is a step function with jumps at $(X_i)_{i \in \{1,\dots,n\}}$. If the breakpoint does not coincide with any datapoint, it has derivative 0 w.r.t. ψ. Under such assumption, the previous expression simplifies into:

$$\begin{aligned}
0 =& -\tilde{\beta}(\psi) \sum_{i=1}^n \mathbb{1}_{X_i \geq \psi} (Y_i - \tilde{\beta}(\psi)\psi) \\
\frac{1}{n} \sum_{i=1}^n \mathbb{1}_{X_i \geq \psi} Y_i =& \tilde{\beta}(\psi)\psi
\end{aligned}$$

So the breakpoint should be such that the average post-breakpoint value equal the fitted plateau.

1.3 Example

Simulate some data:

```
library(lmbreak)

set.seed(10)
df10 <- simBreak(c(1, 100), breakpoint = c(0,2,4),
                 slope = c(1,0), sigma = 0.05)
```

Automatic fit of the breakpoint model:

```
e.lmbreak10 <- lmbreak(Y ~ 0 + bp(X, pattern = "10", start = c(2)),
                       data = df10)
logLik(e.lmbreak10)
```

```
[1] 161.5047
```

Estimates:

```
model.tables(e.lmbreak10)
```

```
          X duration intercept    slope
1 0.05773562 1.940347 0.0579186 1.003169
2 1.99808295 1.847127 2.0044152 0.000000
3 3.84520966       NA 2.0044146       NA
```

OLS:

```
XX <- df10$X - pmax(df10$X-coef(e.lmbreak10),0)
solve(t(XX) %*% XX) %*% t(XX) %*% df10$Y
```

```
[,1]
[1,] 1.003169
```

Explicit OLS:

```
sum((df10$X < coef(e.lmbreak10))*df10$X*df10$Y)/sum((df10$X < coef(e.
    lmbreak10))*df10$X^2)
```

```
[1] 1.003169
```

Full likelihood:

```
calcLogLik <- function(theta){
  beta <- theta["beta"]
  psi <- theta["psi"]
  sigma <- exp(theta["logsigma"])
  out <- NROW(df10)/2 * log(2*pi) + NROW(df10)/2 * log(sigma) + sum((df10$
    Y - beta * df10$X + beta * pmax(df10$X - psi,0))^2)/(2*sigma)
  return(unname(out))
}
## By default optim performs minimization
theta.init <- c(beta = 0, psi = 1, logsigma = 0.5)
res.opt <- optim(par = theta.init, fn = calcLogLik)
c(res.opt$par, value = res.opt$value)
```

```
    beta          psi     logsigma        value
1.003165    1.998090   -6.067996 -161.504705
```

We retrieve the estimate and log-likelihood of `lmbreak`.

2 Mixed model, single breakpoint, single slope

2.1 Example

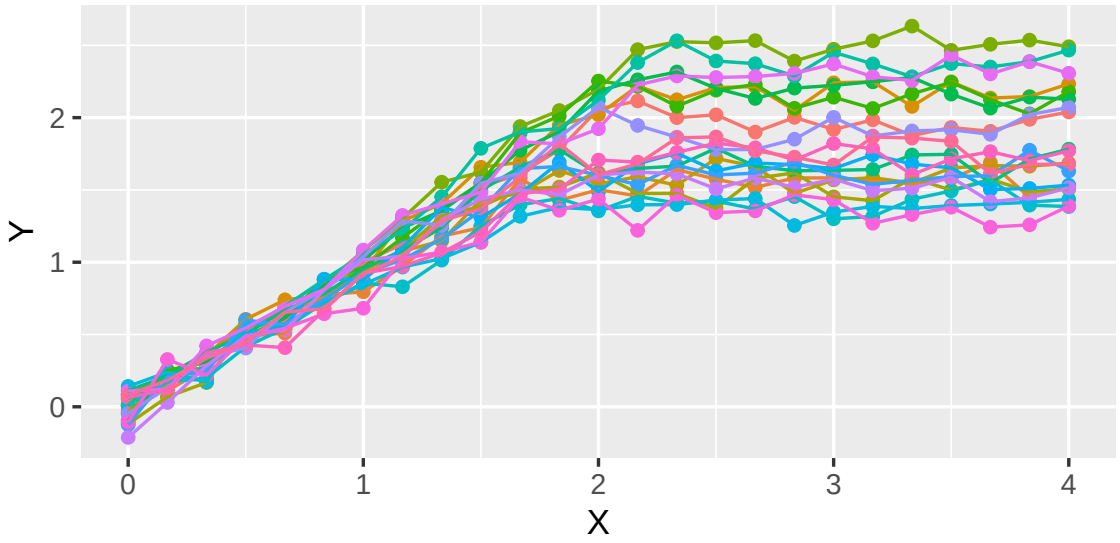
We will consider a dataset of 20 subjects with 25 observations each:

```
library(lmbreak)
library(mvtnorm)

## data generating parameters
n.subject <- 20
sigma2T <- 0.005
betaT <- 1
psiT <- 2
MtauT <- tcrossprod(c(0.2,0.1)) * matrix(c(1,0.5,0.5,1),2,2)
seqTime <- seq(0,4,length.out=25)

set.seed(10)
RE <- rmvnorm(n.subject, mean = c(psiT,betaT), sigma = MtauT)

ls.sim <- lapply(1:n.subject, function(iS){ ## iS <- 1
  iSim <- simBreak(c(1, length(seqTime)),
                   breakpoint = c(min(seqTime),RE[iS,1],max(seqTime)),
                   slope = c(RE[iS,2],0), sigma = sqrt(sigma2T),
                   rdist.X = seqTime)
  iSim$id <- iS
  iSim
})
df.sim <- do.call(rbind,ls.sim)
```



2.2 Direct optimisation (bivariate integral)

2.2.1 Theory

Consider a set of n subjects each undergoing T measurements at equally spaced timepoints $1, \ldots, T$ of an outcome variable Y. We consider the following random slope and random breakpoint model:

$$Y_{i,t} = \beta_i t - \beta_i (t - \psi_i)_+ + \varepsilon_{i,t}$$

where $\beta_i = \beta + u_i$ and $\psi_i = \psi + v_i$ and:

$$\begin{bmatrix} u_i \\ v_i \\ \varepsilon_{i,t} \end{bmatrix} = \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} \tau_{u,u} & \tau_{u,v} & 0 \\ \tau_{u,v} & \tau_{v,v} & 0 \\ 0 & 0 & \sigma^2 \end{bmatrix} \right)$$

We further denote $\Sigma_\tau = \begin{bmatrix} \tau_{u,u} & \tau_{u,v} \\ \tau_{u,v} & \tau_{v,v} \end{bmatrix}$ whose inverse is $\Sigma_\tau^{-1} = \begin{bmatrix} w_{u,u} & w_{u,v} \\ w_{u,v} & w_{v,v} \end{bmatrix} = \frac{1}{\tau_{u,u}\tau_{v,v} - \tau_{u,v}^2} \begin{bmatrix} \tau_{v,v} & -\tau_{u,v} \\ -\tau_{u,v} & \tau_{u,u} \end{bmatrix}$.
We now can re-write the model as:

$$\begin{aligned} Y_{i,t} =& \beta(t - (t - \psi - v_i)_+) + u_i(t - (t - \psi - v_i)_+) + \varepsilon_{i,t} \\ =& \mu(t, \beta, \psi, u_i, v_i) + \varepsilon_{i,t} \end{aligned}$$

where $\mu(t, \beta, \psi, u_i, v_i) = \beta t + u_i t$ when $t \leq \psi + v_i$
and $\mu(t, \beta, \psi, u_i, v_i) = \beta(\psi + v_i) + u_i(\psi + v_i)$ otherwise.
Introduce $\Theta = (\beta, \psi, \tau_{u,u}, \tau_{u,v}, \tau_{v,v}, \sigma^2)$, we can express the likelihood relative to n iid subjects as:

$$\mathcal{L}(\Theta) = \prod_{i=1}^{n} \prod_{t=1}^{T} \int_{u_i=-\infty}^{+\infty} \int_{v_i=-\infty}^{+\infty} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(Y_i - \mu(t, \beta, \psi, u_i, v_i))^2}{2\sigma^2} \right) f_\tau(u, v) du_i dv_i$$

$$\begin{aligned} \text{where } f_\tau(u, v) =& \frac{1}{2\pi \left|\Sigma_\tau\right|^{1/2}} \exp\left(-\frac{1}{2} \begin{bmatrix} u & v \end{bmatrix} \Sigma_\tau^{-1} \begin{bmatrix} u \\ v \end{bmatrix} \right) \\ =& \frac{1}{2\pi\sqrt{\tau_{u,u}\tau_{v,v} - \tau_{u,v}^2}} \exp\left(-\frac{u^2 w_{u,u} + 2uvw_{u,v} + v^2 w_{v,v}}{2} \right) \end{aligned}$$

One could evaluate this log-likelihood using numerical integration but this will typically be very slow:

- e.g. for a single subject at a single timepoint:

```
system.time(
  resIntegral2 <- logLik_mixed10(
    Y = df.sim[df.sim$id==1,"Y"], t = df.sim[df.sim$id==1,"X"],
    beta = betaT, psi = psiT, tau_uu = MtauT, sigma2 = sigma2T,
    integration = "bivariate")
```

```
)
```

```
bruger   system forløbet
 35.35     2.81    39.87
```

```
head(resIntegral2)
```

```
  integral1 error1 integral2 error2 integral        error    logLik
1        NA     NA        NA     NA 3.126521 3.086796e-05 1.1399209
2        NA     NA        NA     NA 3.923091 3.868127e-05 1.3668797
3        NA     NA        NA     NA 3.301497 3.292422e-05 1.1943759
4        NA     NA        NA     NA 2.990112 2.963236e-05 1.0953108
5        NA     NA        NA     NA 2.300690 2.294792e-05 0.8332089
6        NA     NA        NA     NA 2.195036 2.160457e-05 0.7861986
```

Since standard optimizers may evaluate the likelihood a large number of times this direct approach is not really feasible (too slow).

2.3 Direct optimisation (nested univariate integrals)

2.3.1 Theory

To simplify the numerical integration for a bivariate integral to univariate integral, we start by splitting second integral depending on whether $t \leq \psi + v_i$ i.e. $v_i \geq t - \psi$:

$$\begin{aligned}
\mathcal{L}(\Theta) =& \prod_{i=1}^{n} \prod_{t=1}^{T} \int_{u_i=-\infty}^{+\infty} \int_{v_i=t-\psi}^{+\infty} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(Y_i - \beta t - u_i t)^2}{2\sigma^2}\right) f_\tau(u_i, v_i) du_i dv_i \\
&+ \prod_{i=1}^{n} \prod_{t=1}^{T} \int_{u_i=-\infty}^{+\infty} \int_{v_i=-\infty}^{t-\psi} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(Y_i - \beta(\psi + v_i) - u_i(\psi + v_i))^2}{2\sigma^2}\right) f_\tau(u_i, v_i) du_i dv_i \\
=& \frac{1}{(2\pi)^{3nT/2} \sigma^{nT} (\tau_{u,u}\tau_{v,v} - \tau_{u,v}^2)^{nT/2}} \prod_{i=1}^{n} \prod_{t=1}^{T} (\mathcal{L}_{1,i,t}(\Theta) + \mathcal{L}_{2,i,t}(\Theta))
\end{aligned}$$

where we introduce the notation:

$$\begin{aligned}
\mathcal{L}_{1,i,t}(\Theta) &= \int_{u_i=-\infty}^{+\infty} \int_{v_i=t-\psi}^{+\infty} \exp\left(-\frac{(Y_i - \beta t - u_i t)^2}{2\sigma^2} - \frac{u_i^2 w_{u,u} + 2u_i v_i w_{u,v} + v_i^2 w_{v,v}}{2}\right) du_i dv_i \\
\mathcal{L}_{2,i,t}(\Theta) &= \int_{u_i=-\infty}^{+\infty} \int_{v_i=-\infty}^{t-\psi} \exp\left(-\frac{(Y_i - \beta(\psi + v_i) - u_i(\psi + v_i))^2}{2\sigma^2} - \frac{u_i^2 w_{u,u} + 2u_i v_i w_{u,v} + v_i^2 w_{v,v}}{2}\right) du_i dv_i
\end{aligned}$$

Using that:

$$\begin{aligned}\int_a^b \exp\left(-\frac{1}{2}(Av_i^2+2Bv_i)\right)dv_i &= \int_a^b \exp\left(-\frac{A}{2}(v_i^2+2\frac{B}{A}v_i)\right)dv_i \\ &= \int_a^b \exp\left(-\frac{A}{2}(v_i+\frac{B}{A})^2+\frac{B^2}{2A}\right)dv_i \\ &= \sqrt{\frac{2\pi}{A}}\exp\left(\frac{B^2}{2A}\right)\frac{1}{\sqrt{2\pi A^{-1}}}\int_a^b \exp\left(-\frac{(v_i^2--\frac{B}{A})^2}{2A^{-1}}\right)dv_i \\ &= \sqrt{\frac{2\pi}{A}}\exp\left(\frac{B^2}{2A}\right)\left(\Phi_{\mu=-\frac{B}{A},\sigma^2=A^{-1}}(b)-\Phi_{\mu=-\frac{B}{A},\sigma^2=A^{-1}}(a)\right)\end{aligned}$$

where Φ_{μ,σ^2} denotes the Cumulative Distribution Function (CDF) of a normal distribution with mean μ and variance σ^2. For the first term, using $a = t-\psi$, $b = +\infty$, $A = w_{v,v}$, and $B = u_i w_{u,v}$, we obtain:

$$\mathcal{L}_{1,i,t}(\Theta) = \sqrt{\frac{2\pi}{w_{v,v}}}\int_{-\infty}^{+\infty}\exp\left(-\frac{(Y_i-\beta t-u_i t)^2}{2\sigma^2}-\frac{u_i^2(w_{u,u}-\frac{w_{u,v}^2}{w_{v,v}})}{2}\right)(1-\Phi\left(\tau_{v,v}(w_{v,v}(t-\psi)+u_i w_{u,v})\right))$$

which can be evaluated using numerical intergration (e.g. `stats::integrate` and `stats::pnorm` - see function next page). A similar approach can be used to evaluate the second term.

$$\mathcal{L}_{2,i,t}(\Theta) = \int_{u_i=-\infty}^{+\infty}\int_{v_i=-\infty}^{t-\psi}\exp\left(-\frac{(Y_i-\psi(\beta+u_i)-v_i(\beta+u_i))^2}{2\sigma^2}-\frac{u_i^2 w_{u,u}+2u_i v_i w_{u,v}+v_i^2 w_{v,v}}{2}\right)du_i dv_i$$

$$=\int_{u_i=-\infty}^{+\infty}\int_{v_i=-\infty}^{t-\psi}\exp\left(-\frac{(Y_i-\psi(\beta+u_i))^2+v_i^2(\beta+u_i)^2-2v_i(\beta+u_i)(Y_i-\psi(\beta+u_i))}{2\sigma^2}-\frac{u_i^2 w_{u,u}+2u_i v_i w_{u,v}+v_i^2 w_{v,v}}{2}\right)$$

Using $a = -\infty$, $b = t-\psi$, $A = w_{v,v}+(\beta+u_i)^2$, and $B = u_i w_{u,v} - h(u_i)$ where $h(u_i) = (\beta+u_i)(Y_i-\psi(\beta+u_i))$, we obtain for $\mathcal{L}_{2,i,t}(\Theta)$:

$$\sqrt{2\pi}\int_{-\infty}^{+\infty}\exp\left(-\frac{(Y_i-\psi(\beta+u_i))^2}{2\sigma^2}-\frac{u_i^2 w_{u,u}-\frac{(u_i w_{u,v}-h(u_i))^2}{w_{v,v}+(\beta+u_i)^2}}{2}\right)\frac{\Phi\left(\tau_{v,v}((w_{v,v}+(\beta+u_i)^2)(t-\psi)+u_i w_{u,v}-h(u_i)\right)}{\sqrt{w_{v,v}+(\beta+u_i)^2}}du_i$$

This approach is (much) more numerically efficient:

```
system.time(
  resIntegral <- logLik_mixed10(Y = df.sim$Y, t = df.sim$X,
                                beta = betaT, psi = psiT, sigma2 = sigma2T
    ,
                                tau_uu = MtauT[1,1], tau_uv = MtauT[1,2],
    tau_vv = MtauT[2,2],
                                integration  = "univariate")
```

```
)
```

```
bruger   system forløbet
  0.30     0.00     0.35
```

```
head(resIntegral)
```

```
    integral1       error1    integral2       error2  integral error    logLik
1 0.002075497 3.177549e-05 2.348028e-12 4.254261e-12 0.2697098    NA -1.310409
2 0.002594158 1.206309e-04 3.027044e-11 5.717863e-11 0.3371095    NA -1.087348
3 0.002168798 1.148973e-05 3.815324e-08 7.213300e-08 0.2818391    NA -1.266419
4 0.001949584 8.319586e-06 3.005641e-08 5.669998e-08 0.2533513    NA -1.372978
5 0.001471287 5.025842e-05 1.462091e-08 2.628091e-08 0.1911949    NA -1.654462
6 0.001374173 1.000105e-04 1.891155e-05 3.606195e-05 0.1810305    NA -1.709090
```

However something is not quite right as the two approaches do not lead to similar results (TO DEBUG)

```
cor(resIntegral$logLik[1:NROW(resIntegral2)],resIntegral2$integral)
```

```
[1] -0.5922597
```